

Getting Started: Wokwi and JSLinux

Script: Preface, Chapter 1.1

with solutions

Learning goals

- Run, modify and share an Arduino project in the Wokwi simulator.
- *Bonus (optional)*: compile and run C programs in JSLinux, plot results in the terminal, and transfer measured data from Wokwi (Serial Monitor) to JSLinux.
- Get to know the course library `code/common/engine_signals.h`, the *virtual engine* used on all following sheets.

The virtual engine

Throughout the exercises we cannot measure on a real engine. Instead, a *virtual engine* generates the sensor signals introduced in Chapter 1 of the script. Exactly the same C model is used in three places:

Where	How
JSLinux	<code>#include "engine_signals.h"</code> in your C program
Wokwi (sketch)	add <code>engine_signals.h</code> as an extra file to the project
Wokwi (custom chip)	part <code>chip-engine-sim</code> : outputs CRANK, CAM, KNOCK, ION, and the ground-truth signals TDC and KNK

The conventions of the model are fixed for all sheets:

- 4-cylinder 4-stroke engine, crank angle $\theta \in [0^\circ, 720^\circ)$, firing order 1–3–4–2, combustion TDC at $120^\circ, 300^\circ, 480^\circ, 660^\circ$.
- 60-2 trigger wheel: tooth $k = 0, \dots, 57$ is HIGH for $\theta \bmod 360^\circ \in [6k^\circ, 6k^\circ + 3^\circ)$; teeth 58 and 59 are missing.
- Camshaft signal HIGH for $0^\circ \leq \theta < 180^\circ$.
- Knock: damped oscillation with modes at 6.5 kHz and 10.5 kHz, decay time 0.8 ms, onset 5–25° after TDC.
- Disturbances: broadband noise, valve impacts (8 kHz, 90° after TDC), low-frequency combustion vibration, a 1 % speed ripple at twice crankshaft frequency.

Exercise 0.1 First steps in Wokwi

WOKWI

Open <https://wokwi.com>, create a new *Arduino Uno* project and replace `sketch.ino` and `diagram.json` by the files in `code/wokwi/sheet00_hello/`.

- Start the simulation, turn the potentiometer and watch the Serial Monitor. Which ADC value corresponds to 2.5 V? What is the voltage resolution (1 LSB) of the 10-bit ADC?
- The sketch toggles pin 13 at every sample. Stop the simulation after about one second, download the logic analyzer recording (`.vcd`) and determine the sampling period. Does it match `TS_US`?
- Change the sampling rate to 1 kHz. Explain why the printed `t_us` values still show regular increments although the `Serial.print` calls take time.
- Wokwi runs in *simulated time*. Compare the simulated time in the status bar with the time on your wall clock. Why is this important when we later measure execution times?

Solution

- a) $V = \text{adc} \cdot 5/1023$, so $2.5 \text{ V} \approx 511/512$. $1 \text{ LSB} = 5/1023 \text{ V} \approx 4.9 \text{ mV}$.
- b) Pin 13 toggles at every sample, i.e. the period of the rectangular signal on D0 is $2T_s = 20 \text{ ms}$; consecutive edges are $T_s = 10 \text{ ms}$ apart.
- c) The loop waits for an absolute deadline `t_next` that is incremented by `TS_US`. The time spent in printing only shortens the waiting time, it does not shift the next deadline (no drift), as long as one loop iteration takes less than T_s . At 1 kHz and 115200 baud a line of about 20 characters takes $\approx 1.7 \text{ ms}$ – longer than T_s ! Then the deadlines are missed and the samples come out late; the `t_us` increments become irregular. Remedy: fewer characters, higher baud rate, or buffer samples and print blocks.
- d) The simulator may run faster or slower than real time. `micros()` and the logic analyzer use simulated time, so timing *inside* the simulation is consistent. Wokwi is not cycle-accurate for all peripherals, therefore measured execution times are only indicative; we count operations instead.

Exercise 0.2 The engine simulator chip**WOKWI**

Every Wokwi project of the following sheets contains the custom chip `engine-sim`. Add the files `engine-sim.chip.c` and `engine-sim.chip.json` from `code/wokwi/chips/` to the Uno project of Exercise 0.1 and add the part `{ "type": "chip-engine-sim", "id": "eng", "attrs": { "vmax": "5", "of` to `diagram.json`.

- a) Connect `eng:CRANK` to logic analyzer D1, `eng:CAM` to D2 and `eng:TDC` to D3 (and `eng:GND` to ground). Record 100 ms at 3000 rpm and view the `.vcd` file (e.g. in PulseView or GTKWave). Identify the missing-tooth gap and verify the tooth period.
- b) How many crank teeth lie between the gap and the first TDC pulse? Compare with the conventions above.
- c) Change the engine speed with the slider of the chip and repeat the tooth-period measurement at 6000 rpm.

Solution

- a) At 3000 rpm one revolution takes 20 ms; the tooth period (rising to rising edge) is $20 \text{ ms}/60 = 333 \mu\text{s}$, across the gap $3 \cdot 333 \mu\text{s} = 1 \text{ ms}$.
- b) The reference edge (first tooth after the gap) is at $\theta = 0^\circ$, TDC of cylinder 1 at 120° , i.e. 20 teeth after the reference edge.
- c) At 6000 rpm: $167 \mu\text{s}$ per tooth, $500 \mu\text{s}$ across the gap.

Deliverables

Screenshot of the Wokwi logic analyzer view (or VCD plot) with the gap marked. Bonus (optional): terminal output of `hello_signals`; the CSV-plot program of 0.B1 d).

Bonus: JSLinux (optional)

The lab exercises above use only Wokwi. Interested students can in addition process the engine signals in plain C on a Linux system that runs in the browser. The JSLinux exercises of the following sheets build on this one.

Exercise 0.B1 First steps in JSLinux**JS LINUX**

Open <https://bellard.org/jslinux> and start the *x86 Alpine Linux* (console) virtual machine.

- a) Check the toolchain: `gcc -version` (or `tcc -v`). Upload `engine_signals.h`, `asciipplot.h`,

`csvio.h` (code/common/) and `code/bonus/jslinux/sheet00/hello_signals.c` with the upload button below the terminal.

```
1 gcc -O2 -o hello_signals hello_signals.c -lm
2 ./hello_signals
```

- b) Explain the plot of the knock signal: where does the burst start, how long does it last, and what is visible before the burst?
- c) Modify the program such that it plots one full engine cycle (720°) at 3000 rpm with `knock_prob = 0`. How many samples are needed at $f_s = 50$ kHz? Identify the valve impacts and the combustion vibration in the plot.
- d) Copy 2–3 seconds of the Wokwi Serial Monitor output from Exercise 0.1 into a file `pot.csv` in JSLinux (`cat > pot.csv`, paste, `Ctrl-D`). Write a short program that reads it with `csv_read()` and plots the voltage column.

Solution

b) The program starts the record at $\theta = 125^\circ$, i.e. 5° after TDC of cylinder 1. The knock onset lies between 5° and 25° ATDC; at 3000 rpm ($18\,000^\circ/\text{s}$) 1° corresponds to $55.6\,\mu\text{s}$, so the burst appears within the first 1.1 ms. It decays with $\tau = 0.8\text{ ms}$ and is practically gone after $3\tau \approx 2.4\text{ ms}$. Before the burst only noise and the slow combustion vibration (a half-sine over 60°) are visible.

c) One cycle lasts $2 \cdot 60/3000\text{ s} = 40\text{ ms}$, i.e. 2000 samples. Valve impacts appear 90° after each TDC (8 kHz bursts at $\theta = 210^\circ, 390^\circ, 570^\circ, 30^\circ$), the combustion vibration as broad positive humps directly after each TDC.

d) Example:

```
1 csv_t d;
2 if (csv_read("pot.csv", &d) == 0) {
3     double *v = csv_col(&d, 3);          /* column "volt" */
4     ap_plot(v, d.rows, "potentiometer voltage");
5     free(v); csv_free(&d);
6 }
```